

MINI PROJECT

SIMPLE CALCULATOR

DONE BY:

Afeia Justin Jose Little Bright (192524494)

Roshnee Devi Kannan (192511002)

Anisha Jenifer J (192511004)

ABSTRACT

This mini project, titled “Simple Calculator Using Object-Oriented Programming in C++”, is designed to demonstrate the implementation of core object-oriented concepts through a practical application. The system performs basic arithmetic operations such as addition, subtraction, multiplication, and division while incorporating advanced programming principles.

The project utilizes inheritance by deriving a Calculator class from a base class, enabling code reusability and structured design. Polymorphism is achieved through the use of virtual functions and base class pointers, allowing dynamic method binding at runtime. Encapsulation is maintained by restricting direct access to data members and providing controlled interaction through member functions. A static data member is used to keep track of the number of objects created, illustrating shared class-level data.

Exception handling is implemented to manage runtime errors, particularly division by zero, ensuring program stability and user safety. Dynamic memory allocation is also employed to demonstrate efficient memory management using constructors and destructors. Overall, this project provides a clear understanding of how object-oriented programming concepts can be applied to build modular, maintainable, and robust applications in C++. It serves as a foundational example for beginners to bridge theoretical knowledge with practical implementation.

INTRODUCTION

A calculator is a basic tool used to perform arithmetic operations quickly and accurately. This project, “Simple Calculator Using Object-Oriented Programming in C++”, is developed to demonstrate fundamental OOP concepts through a practical application.

The program performs operations such as addition, subtraction, multiplication, and division using classes and objects. It incorporates key concepts like inheritance, polymorphism, encapsulation, and exception handling to ensure structured and efficient code.

This project helps in understanding how theoretical OOP concepts can be applied to build simple and reliable real-world applications.

OBJECTIVES

The main objectives of this project are:

- To develop a simple calculator that performs basic arithmetic operations such as addition, subtraction, multiplication, and division.
- To implement object-oriented programming concepts like classes, inheritance, polymorphism, and encapsulation in C++.
- To handle runtime errors effectively using exception handling mechanisms.
- To understand the use of static members and dynamic memory allocation.
- To design a structured and user-friendly program for basic calculations.

This project aims to strengthen the understanding of OOP principles through practical implementation.

SCOPE OF THE PROJECT

The scope of this project is limited to developing a simple console-based calculator that performs basic arithmetic operations such as addition, subtraction, multiplication, and division.

This project is mainly intended for educational purposes to demonstrate the implementation of object-oriented programming concepts in C++. It can be used by beginners to understand how OOP principles are applied in real-world applications.

Although the current system performs only basic operations, it can be further enhanced by adding advanced mathematical functions, graphical user interfaces, and additional features to make it more interactive and user-friendly.

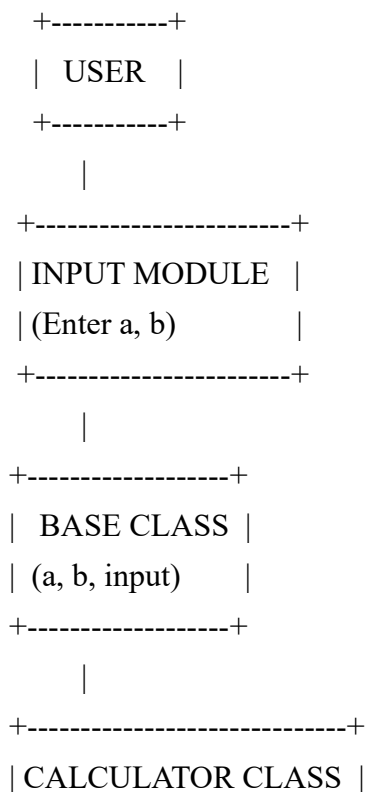
OOP CONCEPTS USED

This project is based on object-oriented programming principles. The following concepts are implemented:

- **Class and Object:** The program uses classes such as *Base* and *Calculator* to define properties and functions. Objects are created to access these functionalities.
- **Inheritance:** The *Calculator* class inherits from the *Base* class, allowing reuse of code and extending functionality.
- **Encapsulation:** Data members are protected and accessed through member functions, ensuring data security and controlled access.
- **Polymorphism:** The use of virtual functions allows dynamic binding, where the appropriate function is called at runtime.
- **Abstraction:** The internal implementation details are hidden, and only necessary functionalities are exposed to the user.
- **Static Members:** A static variable is used to keep track of the number of objects created.
- **Exception Handling:** The program handles errors such as division by zero using try-catch blocks, improving reliability.

These concepts make the program modular, reusable, and easy to maintain.

SYSTEM ARCHITECTURE



```

| + - * / operations      |
| (Exception handling)    |
+-----+
|
+-----=====+
| OUTPUT MODULE |
| Display result      |
+-----+
|
|
v
+-----+
|  END  |
+-----+

```

PROGRAM

```

#include <iostream>

using namespace std;

// Base Class
class Base
{
protected:
    int a, b;
    static int count;

public:
    // Constructor
    Base()
    {
        cout << "Constructor Called\n";
        count++;
    }

    // Virtual Destructor

```

```

virtual ~Base()
{
    cout << "Destructor Called\n";
}

void input()
{
    cout << "Enter two numbers: ";
    cin >> a >> b;
}

static void showCount()
{
    cout << "Objects Created: " << count << endl;
}

virtual void display()
{
    cout << "Base Display\n";
}
};

int Base::count = 0;

// Derived Class
class Calculator : public Base
{
public:
    int add()
    {

```

```
    return a + b;
}
```

```
int sub()
{
    return a - b;
}
```

```
int mul()
{
    return a * b;
}
```

```
float div()
{
    if (b == 0)
        throw "Division by zero!";
    return (float)a / b;
}
```

```
void display()
{
    cout << "Calculator Object\n";
}

};
```

```
int main()
{
    // Object 1
    Calculator c1;
```

```
c1.input();

cout << "\nResults for Object 1\n";
cout << "Addition: " << c1.add() << endl;
cout << "Subtraction: " << c1.sub() << endl;
cout << "Multiplication: " << c1.mul() << endl;
```

```
try
{
    cout << "Division: " << c1.div() << endl;
}
catch (const char *msg)
{
    cout << msg << endl;
}
```

```
c1.display();
```

```
cout << "\n-----\n";
```

```
// Object 2
```

```
Calculator c2;
```

```
c2.input();
```

```
cout << "\nResults for Object 2\n";
cout << "Addition: " << c2.add() << endl;
cout << "Subtraction: " << c2.sub() << endl;
cout << "Multiplication: " << c2.mul() << endl;
```

```
try
```

```

    {
        cout << "Division: " << c2.div() << endl;
    }
    catch (const char *msg)
    {
        cout << msg << endl;
    }

    c2.display();

    // Show number of objects
    Base::showCount();

    return 0;
}

```

OUTPUT

```

Constructor Called
Enter two numbers: 4 5

Results for Object 1
Addition: 9
Subtraction: -1
Multiplication: 20
Division: 0.8
Calculator Object

-----
Constructor Called
Enter two numbers: 5 6

Results for Object 2
Addition: 11
Subtraction: -1
Multiplication: 30
Division: 0.833333
Calculator Object
Objects Created: 2
Destructor Called
Destructor Called

```

COMPLEXITIES

Time Complexity: $O(1)$

Reason:

- Arithmetic operations like addition, subtraction, multiplication, and division take constant time.
- Input, output, and display functions execute in constant time.

Space Complexity: $O(1)$

Reason:

- Only a fixed number of variables (a, b, result) are used.
- No additional data structures like arrays or vectors are required.

Best Case

All operations are performed with valid input ($b \neq 0$).

Time Complexity: $O(1)$

Average Case

Normal execution with valid inputs and standard operations.

Time Complexity: $O(1)$

Worst Case

Division by zero occurs and exception handling is triggered.

Time Complexity: $O(1)$

REFERENCES

1. Herbert Schildt, *C++: The Complete Reference*, McGraw Hill Education.
2. Stanley B. Lippman, Josée Lajoie, Barbara E. Moo, *C++ Primer*, Addison-Wesley.
3. Yashavant P. Kanetkar, *Let Us C++*, BPB Publications.
4. <https://www.geeksforgeeks.org/c-plus-plus/>